



Chapter 4

Data movement Instructions and Addressing



The 8086 microprocessor supports 8 types of instructions:

- Data Transfer Instructions
- Register Addressing
- Immediate Addressing
- Direct Data Addressing
- Register Indirect Addressing
- Base-Plus-Index Addressing
- Register Relative Addressing



INTRODUCTION

- An **instruction** is a **binary pattern designed** inside a **microprocessor to perform a specific function**.
- **Instruction** is a **command use by a processor** to perform a specific operation.
- The **entire group of instructions** that a **microprocessor** supports is called Instruction Set.
- 8086 has more than **20,000 instructions**.

Data Transfer Instruction

Data transfer instruction are those which are used to **movement of the data** from **one location** to another **location** such as **memory** to **microprocessor**, **microprocessor** to port or memory or vice versa.

OR

The **data-transfer functions** provide **the ability to move data** either **between** its **internal registers** or between an internal register **and a storage location in memory**.

Data Transfer Instructions

These instructions are used to transfer the data from the source operand to the destination operand.

Instruction to transfer a word

- ✓ **MOV** – Used to copy the byte or word from the provided source to the provided destination.
- ✓ **PUSH** – Used to put a word at the top of the stack.
- ✓ **POP** – Used to get a word from the top of the stack to the provided location.
- ✓ **PUSHA** – Used to put all the registers into the stack.
- ✓ **POPA** – Used to get words from the stack to all registers.
- ✓ **XCHG** – Used to exchange the data from two locations.
- ✓ **XLAT** – Used to translate a byte in AL using a table in the memory.

Cont..

The **Data-transfer functions** include –

- **MOV** (Move byte or word)
- **PUSH** (Push to Stack)
- **POP** (Pop from Stack)
- **XCHG** (Exchange byte or word or exchanges contents of D and S)
- **IN** (Input to the port or copies a byte or word from S to D)
- **Output** (Output to the port or copies a byte or word from D to S)
- **PUSHA** (put all the registers into the stack)
- **POPA** (gets words from the stack to all registers)
- **PUSHF** (copies the flag register at the top of the stack)
- **POPF** (copies a word at the top of the stack to the flag register)

Data Movement Instructions MOV

❖ MOV Des, Src

- ❖ **Src** operand can be register, memory location or immediate operand.
- ❖ **Des** can be register or memory operand.
- ❖ Both **Src** and **Des** cannot be memory location at the same time.
- ❖ **Example:**

MOV CX, 037AH

MOV AL, BL

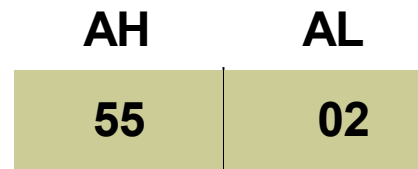
MOV BX, [0301H]

Data Movement Instructions MOV

MOV Des, Src

Example: DS=ACDEH, SI=0724H

MOV AX, [SI]



072CH	18H	AD5FCH
072BH	A3H	AD5FBH
072AH	7EH	AD5FAH
0729H	69H	AD5F9H
0728H	AAH	AD5F8H
0727H	2EH	AD5F7H
0726H	00H	AD5F6H
0725H	55H	AD5F5H
0724H	02H	AD5F4H
0723H	72H	AD5F3H
0722H	11H	AD5F2H

❖ **Physical Address**
= ACEDx10H + 0724H
= AD5F4H, AD5F5H

AX=5502H

Data Movement Instructions PUSH and POP

PUSH and POP Instructions

- ❖ Both are important to store and retrieve data from the LIFO structure (stack).

- ❖ PUSH Operand

- ❖ It pushes the operand into top of stack.

- ❖ Example:

PUSH BX

- ❖ POP Des

- ❖ It pops the operand from top of stack to Des.

- ❖ Example:

POP AX

Data Movement Instructions PUSH

PUSH:

- ❑ In 8088 and 8086 PUSH always transfer 2 bytes of data to the stack.

- ❑ It has the following syntax:

PUSH src

- ❑ **Src** could be any 2 byte register or memory location or immediate value.

Instructions for input and output port transfer

- ✓ **IN:** Used to read a byte or word from the provided port to the accumulator.
- ✓ **OUT:** Used to send out a byte or word from the accumulator to the provided port.

Instructions to transfer the address

- ✓ **LEA:** Used to load the address of operand into the provided register.
- ✓ **LDS:** Used to load DS register and other provided register from the memory
- ✓ **LES:** Used to load ES register and other provided register from the memory.

Data Movement Instructions LEA

The sum generated by this instruction

LEA SI, [BX+DI]

is a modulo-64K sum drops any carry out of the
16-bit result

□ **Example:**

LEA SI, [BX+DI]

If BX = 1000H, DI = FF00H, SI = 0F00H

□ **Solution:**

SI=BX+DI

=1000+FF00=0F00 H instead of 10F00H

Data Movement Instructions LEA

❖ Load-Effective Address (LEA)

- ❖ **LEA** stands for load effective address.

LEA Register, Src

- ❖ It loads a 16-bit register with the offset address of the data specified by the Src.

LEAAX, NUMB

- ❖ **AX** ← operand offset address **MUMB**

Data Movement Instructions LEA

□ Example:

LEA BX, [DI] ; loads offset address
specified by [DI] (contents
of DI) into BX register

MOV BX, [DI] ; loads the data stored at
the memory location
addressed by [DI] into BX
register

MOV BX, DI
= LEA BX, [DI]

Data Movement Instructions LDS

❖ LDS Des, Src

- ❖ It loads 32-bit pointer from memory source to destination register and DS.
- ❖ The offset is placed in the destination register and the segment is placed in DS.
- ❖ To use this instruction the word at the lower memory address must contain the offset and the word at the higher address must contain the segment.

Arithmetic instructions in 8086 microprocessor

Arithmetic Instructions are the instructions which perform basic arithmetic operations such as addition, subtraction and a few more. Unlike in 8085 microprocessor, in 8086 microprocessor the destination operand need not be the accumulator.

- **ADD** (Addition)
- **ADC** (Addition with carry)
- **SUB** (Subtraction)
- **SBB** (Subtraction with Borrow)
- **MUL** (Multiplication)

Data Movement Instructions LDS

- ❖ LDS Des, Src

- ❖ Example:

LDS BX, [DI]

- ❖ If DS= 1000H, DI = 1000H

- ❖ The LDS BX, [DI] instruction loads register BX from addresses 11000H and 11001H and register DS from locations 11002H and 11003H. This instruction is shown at the point just before DS changes to 3000H and BX changes to 127AH.

Data Movement Instructions XCHG

XCHG Des, Src

- ❖ This instruction exchanges Src with Des.
- ❖ It Exchanges contents of a register with any other register or memory location.
- ❖ It **can not** exchange **segment registers**.
- ❖ It **can not** exchange **two memory locations** directly.
- ❖ Use any addressing mode except immediate.
- ❖ **Example:**

XCHG DX, AX

DX= 3000H, AX = 1000H

- ❖ **After Execution this Instruction**
DX= 1000H, AX = 3000H

Data Movement Instructions IN and OUT

❖ IN Accumulator, Port Address

- ❖ This instruction transfers the operand from specified port to accumulator register.

IN AX, 0028 H

- ❖ An IN instruction transfers data from an external I/O device into AL or AX

❖ OUT Port Address, Accumulator

- ❖ This instruction transfers the operand from accumulator to specified port.

OUT 0028 H, AX

- ❖ An OUT transfers data from AL or AX to an external I/O device.

Data Movement Instructions IN and OUT

- ☐ **IN & OUT instructions perform I/O operations.**
 - ☐ **Contents of AL or AX are transferred only between I/O device and microprocessor.**
 - ☐ **Two forms of I/O device (port) addressing:**
 - ☐ **Fixed-port addressing allows data transfer between AL, or AX using an 8-bit I/O port address.**
 - ☐ **Port number follows the instruction's opcode.**
- IN AX, 0028 H**
OUT 0028 H, AX

Data Movement Instructions IN and OUT

- Variable-port addressing allows data transfers between AL or AX and a 16-bit port address.
- The I/O port number is stored in register DX, which can be changed (varied) during the execution of a program.

MOV DX, 0028H

IN AX, DX

OUT DX, AX

Arithmetic Instructions

These instructions are used to perform arithmetic operations like addition, subtraction, multiplication, division, etc.

Instructions to perform addition

- ✓ **ADD**: Used to add the provided byte to byte/word to word.
- ✓ **ADC**: Used to add with carry.
- ✓ **INC**: Used to increment the provided byte/word by 1.
- ✓ **AAA**: Used to adjust ASCII after addition.
- ✓ **DAA**: Used to adjust the decimal after the addition/subtraction operation.

Instructions to perform subtraction

- ✓ **SUB**: Used to subtract the byte from byte/word from word.
- ✓ **SBB**: Used to perform subtraction with borrow.
- ✓ **DEC**: Used to decrement the provided byte/word by 1.
- ✓ **CMP**: Used to compare 2 provided byte/word.
- ✓ **AAS**: Used to adjust ASCII codes after subtraction.
- ✓ **DAS**: Used to adjust decimal after subtraction.

Instruction to perform multiplication

- ✓ **MUL** – Used to multiply unsigned byte by byte/word by word.
- ✓ **IMUL(integer MUL)** – Used to multiply signed byte by byte/word by word.
- ✓ **AAM** – Used to adjust ASCII codes after multiplication.

Instructions to perform division

- ✓ **DIV** – Used to divide the unsigned word by byte or unsigned double word by word.
- ✓ **IDIV** – Used to divide the signed word by byte or signed double word by word.
- ✓ **AAD** – Used to adjust ASCII codes after division.
- ✓ **CBW** – Used to fill the upper byte of the word with the copies of sign bit of the lower byte.
- ✓ **CWD** – Used to fill the upper word of the double word with the sign bit of the lower word.

Logical instructions in 8086 microprocessor

Logical instructions are the instructions which perform basic logical operations such as AND, OR, etc. In 8086 microprocessor, the destination operand need not be the accumulator.

- NOT
- AND
- OR
- XOR

Bit Manipulation Instructions

These instructions are used to perform operations where data bits are involved, i.e. operations like logical, shift, etc.

Instructions to perform logical operation

- ✓ **NOT**: Used to invert each bit of a byte or word.
- ✓ **AND**: Used for adding each bit in a byte/word with the corresponding bit in another byte/word.
- ✓ **OR**: Used to multiply each bit in a byte/word with the corresponding bit in another byte/word.
- ✓ **XOR**: Used to perform Exclusive-OR operation over each bit in a byte/word with the corresponding bit in another byte/word.
- ✓ **TEST**: Used to add operands to update flags, without affecting operands.

Program Execution Transfer Instructions (Branch and Loop Instructions)

These instructions are used to transfer/branch the instructions during an execution. There are two types of branching instructions.

The unconditional branch and conditional branch.

The Unconditional Branch instructions are-

- **CALL** – Used to call a procedure and save their return address to the stack.
- **RET** – Used to return from the procedure to the main program.
- **JMP** – Used to jump to the provided address to proceed to the next instruction.
- **LOOP** - Used to loop a group of instructions until the condition satisfies, i.e., $CX = 0$

Conditional Branch instructions are-

- **JC** (Used to jump if carry flag $CY = 1$)
- **JNC** (Used to jump if no carry flag ($CY = 0$))
- **JE/JZ** (Used to jump if equal/ jump if zero flag $ZF = 1$)
- **JNE/JNZ** (Used to jump if not equal/zero flag $ZF = 0$)
- **JO** (Used to jump if overflow flag $OF = 1$)
- **JNO** (Used to jump if no overflow flag $OF = 0$)
- **LOOPE/ LOOPZ** (Used to loop a group of instructions till it satisfies $ZF = 1$ & $CX = 0$)
- **LOOPNE/ LOOPNZ** (Used to loop a group of instructions till it satisfies $ZF = 0$ & $CX = 0$)

Iteration Control Instructions

These instructions are used to execute the given instructions for number of times.

- ✓ **LOOP** – Used to loop a group of instructions until the condition satisfies, i.e., $CX = 0$
- ✓ **LOOPE/LOOPZ** – Used to loop a group of instructions till it satisfies $ZF = 1 \ \& \ CX = 0$
- ✓ **LOOPNE/LOOPNZ** – Used to loop a group of instructions till it satisfies $ZF = 0 \ \& \ CX = 0$
- ✓ **JCXZ** – Used to jump to the provided address if $CX = 0$

Interrupt Instructions

These instructions are used to call the interrupt during program execution.

- ✓ **INT** – Used to interrupt the program during execution and calling service specified.
- ✓ **INTO** – Used to interrupt the program during execution if $OF = 1$
- ✓ **IRET** – Used to return from interrupt service to the main program

Register Addressing

Register Addressing

- The most common form of data addressing.
 - once register names learned, easiest to apply.
- The microprocessor contains these 8-bit register names used with register addressing: AH, AL, BH, BL, CH, CL, DH, and DL.
- 16-bit register names: AX, BX, CX, DX, SP, BP, SI, and DI.

Cont.

.

	Registers
8-bit	AH, AL, BH, BL, CH, CL, DH, DL
16-bit	AX, BX, CX, DX, SP, BP, DI, SI
32-bit	EAX, EBX, ECX, EDX, ESP, EBP, EDI, ESI

Data-Addressing Modes

• Register addressing	• Transfers a copy of a byte or word from the source register or memory location to the destination register or memory location.
• Immediate addressing	• Transfers the source-immediate byte or word of data into the destination register or memory location.
• Direct addressing	• Moves a byte or word between a memory location and a register. The instruction set does not support a memory-to-memory transfer, except for the MOVS instruction.
• Register indirect addressing	• Transfers a byte or word between a register and a memory location addressed by an index or base register. The index and base registers are BP, BX, DI, and SI.
• Base-plus-index addressing	• Transfers a byte or word between a register and the memory location addressed by a base register (BP or BX) plus an index register (DI or SI).
• Register relative addressing	• Moves a byte or word between a register and the memory location addressed by an index or base register plus a displacement.
• Base relative-plus-index addressing	• Transfers a byte or word between a register and the memory location addressed by a base and an index register plus a displacement.
• Scaled-index addressing	• Is available only in the 80386 through the Pentium 4 microprocessor. The second register of a pair of registers is modified by the scale factor of 2X, 4X, or 8X to generate the operand memory address.

Register Addressing

Register addressing is the most common form of data addressing and, once the register names are learned, is the easiest to apply.

- **Examples of the register-addressed instructions.**

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV AL, BL	8-bits	Copies BL into AL
MOV CH, CL	8-bits	Copies CL into CH
MOV AX, CX	16-bits	Copies CX into AX
MOV SP, BP	16-bits	Copies BP into SP
MOV BX, ES	16-bits	Copies ES into BX
MOV ECX, EBX	32-bits	Copies EBX into ECX
MOV ESP, EDX	32-bits	Copies EDX into ESP
MOV ES, DS	—	Not allowed (segment-to-segment)
MOV BL, DX	—	Not allowed (mixed sizes)

Immediate Addressing

Immediate Addressing

- Term *immediate* implies that data immediately follow the hexadecimal opcode in the memory.
 - immediate data are constant data
 - data transferred from a register or memory location are variable data
- Immediate addressing operates upon a byte or word of data.
- Figure 3–4 shows the operation of a MOV EAX,13456H instruction.

Immediate Addressing

The term *immediate* implies that the data immediately follow the hexadecimal opcode in the memory. Also note that immediate data are constant data, while the data transferred from a register are variable data.

- **Examples of the immediate-addressed instructions.**

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV BL, 44	8-bits	Copies a 44 decimal (2CH) into BL
MOV AX, 44H	16-bits	Copies a 0044H into AX
MOV CH, 100	8-bits	Copies a 100 decimal (64H) into CH
MOV AL, 'A'	8-bits	Copies an ASCII A into AL
MOV AX, 'AB'	16-bits	Copies an ASCII BA into AX
MOV CL, 11001110B	8-bits	Copies a 11001110 binary into CL
MOV EBX, 12340000H	32-bits	Copies a 12340000H into EBX
MOV ESI, 12	32-bits	Copies a 12 decimal into ESI
MOV EAX, 100B	32-bits	Copies a 100 binary into EAX

Direct Data Addressing

Direct Data Addressing

- Applied to many instructions in a typical program.
- Two basic forms of direct data addressing:
 - direct addressing, which applies to a MOV between a memory location and AL, AX, or EAX
 - displacement addressing, which applies to almost any instruction in the instruction set
- Address is formed by adding the displacement to the default data segment address or an alternate segment address.

Direct Addressing

- Direct addressing with a MOV instruction transfers data between a memory location, located within the data segment, and the AL (8-bit), AX (16-bit), or EAX (32-bit) register.
 - usually a 3-byte long instruction
- MOV AL,DATA loads AL from the data segment memory location DATA (1234H).
 - DATA is a symbolic memory location, while 1234H is the actual hexadecimal location

Example

Cont..

segment

```
DATA1 DB 10H ; place 10H in DATA1
DATA2 DB 0 ; place 0 in DATA2
DATA3 DW 0 ; place 0 in DATA3
DATA4 DW 0AAAAH ; place AAAAH in DATA4

.MODEL SMALL ; select SMALL model
.DATA ; indicate start of DATA

.CODE ; indicate start of CODE segment
.STARTUP ; indicate start of program
MOV AL, DATA1 ; copy DATA1 to AL
MOV AH, DATA2 ; copy DATA2 to AH
MOV DATA3, AX ; save AX at DATA3
MOV BX, DATA4 ; load BX with DATA4

.EXIT ; exit to DOS
END ; end file
```

Register Indirect Addressing

Register Indirect Addressing

- Allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI, and SI.
- In addition, 80386 and above allow register indirect addressing with any extended register except ESP.
- In the 64-bit mode, the segment registers serve no purpose in addressing a location in the flat model.

Register Indirect Addressing

Register indirect addressing allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI, and SI.

- **Examples of the Register indirect addressed instructions.**

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CX, [BX]	16-bits	Copies the word contents of the data segment memory location address by BX into CX
MOV [BP], DL	8-bits	Copies DL into the stack segment memory location addressed by BP
MOV [DI], BH	8-bits	Copies BH into the data segment memory location addressed by DI
MOV [DI], [BX]	—	Memory-to-memory moves are not allowed except with string instructions
MOV AL, [EDX]	8-bits	Copies the byte contents of the data segment memory location addressed by EDX into AL
MOV ECX, [EBX]	32-bits	Copies the doubleword contents of the data segment memory location addressed by EBX into ECX

Base-Plus-Index Addressing

Base-Plus-Index Addressing

- Similar to indirect addressing because it indirectly addresses memory data.
- The base register often holds the beginning location of a memory array.
 - the index register holds the relative position of an element in the array
 - whenever BP addresses memory data, both the stack segment register and BP generate the effective address

Base-Plus-Index Addressing

Base-plus-index addressing is similar to indirect addressing because it indirectly addresses memory data.

In the 8086 through the 80286, this type of addressing uses one base register (BP or BX), and one index register (DI or SI) to indirectly address memory.

- **Examples of the Base-Plus-Index addressed instructions.**

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CX,[BX+DI]	16-bits	Copies the word contents of the data segment memory location address by BX plus DI into CX
MOV CH,[BP+SI]	8-bits	Copies the byte contents of the stack segment memory location addressed by BP plus SI into CH
MOV [BX+SI],SP	16-bits	Copies SP into the data segment memory location addresses by BX plus SI
MOV [BP+ DI],AH	8-bits	Copies AH into the stack segment memory location addressed by BP plus DI

Example

	.MODEL SMALL	; select SMALL model
	.DATA	; start of DATA segment
ARRAY DB	16 DUP (?)	; setup ARRAY
element 10H	DB 29H	; sample data at
	DB 30 DUP (?)	
segment	.CODE	; start of CODE
program	.STARTUP	; start of
	MOV BX, OFFSET ARRAY	; address ARRAY
	MOV DI, 10H	; address element 10H
	MOV AL, [BX+DI]	; get element 10H
	MOV DI, 20H	; address element 20H
	MOV [BX+DI], AL	; save in element 20H
	.EXIT	; exit to DOS
	END	; end of file

Register Relative Addressing

Register Relative Addressing

- Similar to base-plus-index addressing and displacement addressing.
 - data in a segment of memory are addressed by adding the displacement to the contents of a base or an index register (BP, BX, DI, or SI)
- Figure 3–10 shows the operation of the MOV AX,[BX+1000H] instruction.
- A real mode segment is 64K bytes long.

Register Relative Addressing

Register relative addressing is similar to base-plus-index addressing and displacement addressing. In register relative addressing, the data in a segment of memory are addressed by adding the displacement to the contents of a base or an index register (BP, BX, DI, or SI).

- **Examples of the register Relative addressed instructions.**

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV AX, [DI+100H]	16-bits	Copies the word contents of the data segment memory location addressed by DI plus 100H into AX
MOV ARRAY[SI], BL	8-bits	Copies BL into the data segment memory location addressed by ARRAY plus SI
MOV LIST[SI+2], CL	8-bits	Copies CL into the data segment memory location addressed by sum of LIST, SI, and 2
MOV DI,SET_IT[BX]	16-bits	Copies the word contents of the data segment memory location addressed by the sum of SET_IT and BX into DI
MOV ARRAY[EBX],EAX	32-bits	Moves EAX into the data segment memory location addressed by the sum of ARRAY and EBX

Iteration Control Instructions

These instructions are used to execute the given instructions for number of times.

- **LOOP** – Used to loop a group of instructions until the condition satisfies, i.e., $CX = 0$
- **LOOPE/LOOPZ** – Used to loop a group of instructions till it satisfies $ZF = 1 \ \& \ CX = 0$
- **LOOPNE/LOOPNZ** – Used to loop a group of instructions till it satisfies $ZF = 0 \ \& \ CX = 0$
- **JCXZ** – Used to jump to the provided address if $CX = 0$



loop Syntax

MOV CX,count

label:

.....

loop label



;sum of the first 100 integers

mov cx,100

mov ax,0

mov bx,1

cont:

add ax,bx

inc bx

loop cont



; A PROGRAM THAT PRINTS HELLO WORLD 10 TIMES.

model small

.stack 100h

.data

greet db "Hello WOrld ",0Ah,0DH,"\$"

.code

main proc

mov ax,@data

mov ds,ax

mov dx,offset greet

mov ah,09h

mov cx,0Ah

here:

int 21h

loop here

endp

end main